CIBERTEC

INSTITUTO DE EDUCACIÓN SUPERIOR PRIVADO CIBERTEC
DIRECCIÓN ACADÉMICA
CARRERAS PROFESIONALES

ESCUELA : TECNOLOGÍAS DE LA INFORMACIÓN

CARRERA : COMPUTACIÓN E INFORMÁTICA

CURSO : (14698) DESARROLLO DE SERVICIOS WEB II

PERÍODO : 2026

CICLO : SEXTO

Instituto de Educación Superior Privado CIBERTEC

Integrantes:

- Israel Angulo Salas
- Jonathan Villanueva Roncal (Coordinador del Grupo)

Profesor:

- Alex Belleza Porras

Caso de Estudio: **FisioMarca – Sistema de Fisioterapia con Microservicios**

Plataforma web para un centro de fisioterapia y rehabilitación en Lima, Perú. Los pacientes consultan tratamientos, arman su carrito, reservan sesiones con horario real y pagan online. La clínica gestiona agenda, fisioterapeutas, salas, historial clínico y pagos desde un panel de administración.

2026

Lima - Perú | Sede - Independencia

INFORME DEL PROYECTO FINAL
SISTEMA DE FISIOTERAPIA

Plataforma web para clientes y administrador de un centro de fisioterapia y rehabilitación

Lima - Perú

1.- CONTENIDO

2.- RESUMEN 2
3.- INTRODUCCIÓN 2
4.- FUNDAMENTACIÓN 3
5.- DIAGNÓSTICO – ANÁLISIS SEPTE 3
5.1 Factor Social 3
5.2 Factor Económico 4
5.3 Factor Tecnológico 4
5.4 Factor Ecológico 4
6.- OBJETIVOS 4
6.1 OBJ 1: Desarrollo de la aplicación web 5
6.2 OBJ 2: Arquitectura, seguridad e integración 5
7.- JUSTIFICACIÓN 5
7.1 Beneficiarios Directos 6
7.2 Beneficiarios Indirectos 6
8.- DEFINICIÓN Y ALCANCE 6
8.1 Descripción del Sistema 6
8.2 Roles del Sistema 6
8.3 Módulos del Sistema 6
8.4 Panel Administrativo 7
8.5 Panel Operativo 7
8.6 Lo que Incluye y No Incluye el Alcance 7
9.- PRODUCTOS Y ENTREGABLES 8
9.1 Aplicación Web 8
9.2 Base de Datos MySQL 8
9.3 Código Fuente y estructura del proyecto 8
9.4 Reportes PDF y Excel 9
10.- ARQUITECTURA Y TECNOLOGÍAS UTILIZADAS 9
10.1 Backend y microservicios 9
10.2 Frontend y presentación 10
10.3 API REST y comunicación 10
10.4 Seguridad 10
10.5 Servicio SOAP, asistente virtual y contenerización 11
11.- CONCLUSIONES 11
12.- RECOMENDACIONES 12
13.- GLOSARIO 12
14.- BIBLIOGRAFÍA 13

15.- ANEXOS 13
15.1 Anexo A: Estructura principal del proyecto 13
15.2 Anexo B: Flujo general de una reserva 13
15.3 Anexo C: Flujo de autenticación con JWT 14
15.4 Anexo D: Evidencias en forma de captura del proyecto inicializado 14

2.- RESUMEN

FisioMarca es una plataforma web desarrollada para un centro de fisioterapia y rehabilitación en Lima, Perú. Los pacientes pueden consultar tratamientos, armar su carrito, reservar sesiones con horario real y realizar el pago en línea, mientras que la clínica gestiona la agenda, los fisioterapeutas, las salas, el historial clínico y los pagos desde un panel de administración.

El proyecto está construido con una arquitectura de microservicios en Java 17 y Spring Boot 4.1, junto con Spring Cloud 2025.1. Cuenta con un servidor Eureka para el registro y descubrimiento de servicios, un API Gateway como punto de entrada único y cinco microservicios de negocio: AUTH-SERVICE, Micro-Catalogo, Micro-Reservas, Micro-Carrito y Micro-Soap. La información se almacena en MySQL 8.4, distribuida en cuatro bases de datos, una por dominio.

El frontend está desarrollado con Next.js 16, React 19 y TypeScript, e incluye el sitio del paciente y el panel del administrador. En el aspecto de seguridad, AUTH-SERVICE emite un token JWT firmado con HS256 que es validado tanto por el Gateway como por cada microservicio, aplicando los roles CLIENTE y ADMINISTRADOR. Los microservicios se comunican internamente mediante OpenFeign y el catálogo también se expone mediante un servicio SOAP.

Como funcionalidades complementarias, el sistema genera reportes en PDF y Excel, ofrece modo claro y oscuro, e incorpora un asistente virtual llamado "Marca". Toda la solución se levanta con un solo comando de Docker Compose. De acuerdo con el README del proyecto, la aplicación está preparada para ejecutarse de forma local y no se encuentra desplegada en la nube.

3.- INTRODUCCIÓN

El presente informe documenta el proyecto desarrollado para el curso Desarrollo de Servicios Web II del programa de Computación e Informática de Cibertec. El proyecto denominado FisioMarca nace como una propuesta para digitalizar la atención de un centro de fisioterapia aplicando una arquitectura de microservicios.

En un centro de fisioterapia es necesario controlar información como los tratamientos ofrecidos, la disponibilidad de horarios, los fisioterapeutas y salas asignados, los pagos de cada sesión y la evolución clínica de los pacientes. Cuando esta información se administra por teléfono, en agendas físicas o en registros separados, aumentan los choques de horario, los pagos sin control y la pérdida del historial de cada paciente.

La aplicación busca solucionar esta problemática mediante una plataforma web que concentra las principales operaciones en un solo sistema. El paciente puede encontrar el tratamiento adecuado a su molestia, reservar en horarios disponibles y pagar sin llamar ni hacer cola. La clínica, por su

parte, organiza la agenda sin choques de horario, controla los pagos pendientes y las cancelaciones, y lleva el historial clínico de cada paciente.

Desde el punto de vista académico, el proyecto permite aplicar Service Discovery con Eureka, API Gateway, seguridad con JWT, comunicación entre servicios con Feign, un servicio SOAP junto a servicios REST y el despliegue local con contenedores Docker.

4.- FUNDAMENTACIÓN

El proyecto se fundamenta en la necesidad de construir una aplicación web distribuida, en la que cada dominio del negocio se implementa como un microservicio independiente con su propia base de datos. La arquitectura separa claramente las responsabilidades: Next.js se encarga de la presentación, el Gateway centraliza el acceso y la validación del token, y cada microservicio concentra la lógica y persistencia de su dominio.

El backend utiliza Java 17, Spring Boot 4.1, Spring Cloud 2025.1 (Eureka, Gateway WebFlux, OpenFeign y LoadBalancer), Spring Security con OAuth2 Resource Server, JJWT, Spring Data JPA con Hibernate, Spring Web Services con JAXB para SOAP, ModelMapper, Lombok y Maven. Todas las respuestas de la API siguen un formato estándar con los campos success, mensaje y data.

El frontend utiliza Next.js 16 con App Router y Turbopack, React 19, TypeScript 5, Tailwind CSS 4, Framer Motion, Recharts, React Hook Form con Zod y Lucide Icons. Para los reportes se utilizan jsPDF y ExcelJS. Las dependencias del frontend se administran con pnpm.

La documentación README establece que el proyecto se ejecuta localmente mediante Docker Compose, que levanta MySQL, los siete módulos del backend y el frontend. Por lo tanto, este informe considera como entregable la aplicación funcional en entorno local y no atribuye al proyecto un despliegue en Azure, Railway, Render u otra plataforma de nube.

5.- DIAGNÓSTICO – ANÁLISIS SEPTE

5.1 Factor Social

La rehabilitación física es una necesidad frecuente en la población, y los pacientes requieren conocer qué tratamiento corresponde a su molestia y en qué horarios pueden ser atendidos. Depender de llamadas o de visitas presenciales para coordinar una cita genera demoras e incomodidad.

El proyecto incorpora un catálogo público de 17 tratamientos organizados en 5 categorías, con búsqueda por nombre y filtro por categoría, además de un asistente virtual que sugiere tratamientos según palabras clave y ofrece un botón de WhatsApp. El paciente puede consultar el estado de sus citas y revisar su evolución mediante un gráfico del dolor en escala EVA.

Oportunidades identificadas:

- Facilitar el acceso a la información de tratamientos sin iniciar sesión.
- Permitir reservar y pagar sin llamar ni hacer cola.
- Brindar al paciente seguimiento de su evolución sesión a sesión.
- Orientar al paciente mediante el asistente virtual "Marca".

5.2 Factor Económico

Un centro de fisioterapia depende de la correcta ocupación de su agenda y del control de sus pagos. El sistema permite pagar en línea (de forma simulada para la demo) con tarjeta, Yape o Plin, y aparta los horarios mientras el paciente completa el pago. Los pagos pendientes que vencen pasan automáticamente al estado Expiradas, liberando la agenda.

El panel de administración muestra métricas de citas, ingresos, tasa de cancelación y tratamientos más pedidos, lo que apoya la toma de decisiones de la clínica.

Oportunidades identificadas:

- Evitar choques de horario y tiempos muertos en la agenda.
- Controlar pagos pendientes, expirados y cancelaciones.
- Conocer los ingresos y los tratamientos más solicitados.
- Obtener reportes exportables en PDF y Excel.

5.3 Factor Tecnológico

El factor tecnológico es el componente central del proyecto. La solución utiliza microservicios Spring Boot registrados en Eureka y expuestos a través de un Gateway que valida el JWT. Los microservicios se comunican entre sí con OpenFeign sin pasar por el Gateway, y el catálogo se expone adicionalmente mediante SOAP. El frontend Next.js consume la API a través del Gateway.

Docker Compose permite levantar toda la solución con un solo comando, sin necesidad de instalar Java, Maven, Node ni MySQL en el equipo. Las imágenes se construyen en múltiples etapas a partir de eclipse-temurin:17-jre-alpine, node:24-alpine y mysql:8.4.

Oportunidades identificadas:

- Separación de responsabilidades por dominio, con una base de datos por microservicio.
- Integración de servicios REST y SOAP en una misma solución.
- Ejecución reproducible en cualquier equipo mediante contenedores.
- Posibilidad de ampliar el asistente con IA mediante una clave opcional.

5.4 Factor Ecológico

El sistema aporta de manera indirecta a la reducción del uso de papel, debido a que la agenda, los pagos, el historial clínico y los reportes se gestionan digitalmente. Los reportes pueden generarse en PDF o Excel y consultarse sin imprimirlos.

Asimismo, al permitir reservar y pagar en línea se reducen los desplazamientos del paciente solo para coordinar una cita. La solución no pretende resolver por sí sola un problema ambiental, pero contribuye a digitalizar procesos administrativos de la clínica.

6.- OBJETIVOS

Los objetivos se plantean considerando el alcance real descrito en el README del proyecto y buscan ser específicos, medibles y alcanzables dentro de un proyecto académico.

6.1 OBJ 1: Desarrollo de la aplicación web

Desarrollar una plataforma web para un centro de fisioterapia que permita al paciente consultar tratamientos, armar su carrito, reservar sesiones y pagar en línea, y a la clínica gestionar agenda, equipo, historial clínico y pagos, integrando microservicios Spring Boot, MySQL y un frontend Next.js.

Criterio	Descripción
Específico	Implementar catálogo, carrito, agendamiento, pago, mis citas, perfil con evolución y panel de administración.
Medible	Contar con las rutas del paciente y del administrador funcionales y probadas a través del Gateway.
Alcanzable	Utilizar Java, Spring Boot, Spring Cloud, MySQL y Next.js.
Relevante	Digitalizar la atención y la gestión de un centro de fisioterapia.
Tiempo	Completar la solución como proyecto del ciclo académico.

6.2 OBJ 2: Arquitectura, seguridad e integración

Implementar una arquitectura de microservicios con Service Discovery, API Gateway, seguridad con JWT y roles, comunicación interna con Feign, un servicio SOAP y la ejecución de todos los componentes con Docker.

Criterio	Descripción
Específico	Incluir Eureka, Gateway, JWT con roles CLIENTE y ADMINISTRADOR, Feign, SOAP y Docker Compose.
Medible	Verificar los servicios registrados en Eureka, las respuestas 401/403 ante accesos indebidos y el WSDL disponible.
Alcanzable	Utilizar Spring Cloud, Spring Security, JJWT, Spring Web Services y Docker.
Relevante	Aplicar los lineamientos de servicios web y microservicios del curso.
Tiempo	Implementado como parte del proyecto.

7.- JUSTIFICACIÓN

FisioMarca se justifica porque concentra en una sola plataforma la información necesaria para la atención fisioterapéutica: tratamientos, agenda, fisioterapeutas, salas, reservas, pagos e historial clínico. La solución permite que el paciente gestione su atención en línea y que la clínica trabaje con información organizada.

Una característica importante es que las reglas de negocio no se dejan únicamente en la interfaz. El sistema aplica restricciones como un máximo de 4 sesiones por compra, 1 sesión por día y un máximo de 4 citas activas por paciente, además de validar el token y los roles en el Gateway y en cada microservicio. Esto reduce el riesgo de operaciones inválidas mediante llamadas directas a la API.

7.1 Beneficiarios Directos

- Pacientes: consultan tratamientos, reservan, pagan, revisan sus citas y su evolución.
- Administradores: gestionan agenda, citas, historial clínico, transacciones, equipo, usuarios y catálogo.
- Fisioterapeutas: cuentan con una agenda organizada por profesional y sala.
- Visitantes: consultan el catálogo y el asistente virtual sin necesidad de registrarse.

7.2 Beneficiarios Indirectos

- El centro de fisioterapia, al contar con información centralizada y métricas de su operación.
- Futuros equipos de desarrollo, que pueden utilizar la estructura de microservicios como base para nuevas funcionalidades.
- La comunidad académica de Cibertec, como referencia de integración entre Spring Cloud, SOAP, Next.js y Docker.

8.- DEFINICIÓN Y ALCANCE

8.1 Descripción del Sistema

FisioMarca es una aplicación web con tres áreas. El área pública permite ver el inicio, los tratamientos y las categorías, e iniciar sesión o registrarse. El área del paciente, protegida por sesión, permite gestionar el carrito, agendar y pagar, revisar sus citas y su perfil. El área de administración, protegida para el rol ADMINISTRADOR, reúne el panel de métricas y la gestión de la clínica.

En el backend, AUTH-SERVICE gestiona usuarios y emite el JWT; Micro-Catalogo administra categorías y tratamientos; Micro-Reservas gestiona agenda, reservas, pagos, equipo e historial clínico; Micro-Carrito administra el carrito y, al confirmarlo, crea la reserva mediante Feign; y Micro-Soap expone el catálogo por SOAP. En el frontend, el archivo src/proxy.ts protege las rutas: sin sesión redirige a /login y cada rol solo entra a sus secciones.

8.2 Roles del Sistema

Rol	Acceso	Principales funcionalidades
ADMINISTRADOR	Panel de administración	Panel de métricas, agenda, citas, historial clínico, transacciones, equipo, usuarios, catálogo, reportes y SOAP Explorer.
CLIENTE	Área del paciente	Perfil y evolución, carrito, agendamiento, pago y mis citas.
Público	Sin autenticación	Inicio, tratamientos, categorías, asistente virtual, login y registro.

8.3 Módulos del Sistema

- Servidor-Eureka (8761): registro y descubrimiento de servicios.
- Gateway (8080): punto de entrada único, rutas y validación del JWT.
- AUTH-SERVICE (8099): login, registro y usuarios; emite el JWT.
- Micro-Catalogo (8090): categorías y tratamientos (REST).

- Micro-Reservas (8092): agenda, reservas, pagos, equipo e historial clínico (REST).
- Micro-Carrito (8093): carrito; al confirmar crea la reserva vía Feign.
- Micro-Soap (8098): catálogo expuesto por SOAP.
- Frontend (3000): sitio del paciente y panel de administración en Next.js.

8.4 Panel Administrativo

El panel administrativo está disponible para el rol ADMINISTRADOR. Presenta métricas de citas, ingresos, tasa de cancelación y tratamientos más pedidos. La agenda se organiza por fisioterapeuta y sala y permite reprogramar citas. El módulo Citas presenta pestañas por estado: pago pendiente, programadas, en atención, atendidas, canceladas y expiradas.

El panel también incluye el historial clínico, las transacciones, la gestión del equipo (fisioterapeutas y salas), usuarios y catálogo, la exportación a PDF y Excel, y un SOAP Explorer para probar el servicio SOAP desde el navegador.

8.5 Panel Operativo

El panel operativo corresponde a la experiencia del paciente. Tras iniciar sesión, el paciente añade sesiones al carrito, accede a Agendar y pagar, elige el día y la hora de cada sesión sobre la disponibilidad real y realiza el pago. Mientras paga, los horarios quedan apartados.

En Mis citas el paciente revisa el estado de cada cita, retoma los pagos pendientes y cancela si lo necesita. En Perfil, la sección Mi evolución muestra un gráfico del dolor en escala EVA sesión a sesión.

8.6 Lo que Incluye y No Incluye el Alcance

Incluido:

- Autenticación JWT (HS256) y autorización por roles CLIENTE y ADMINISTRADOR.
- Catálogo de 17 tratamientos en 5 categorías con búsqueda y filtro.
- Carrito, agendamiento sobre disponibilidad real y pago online simulado.
- Reglas de negocio de sesiones y citas activas.
- Agenda por fisioterapeuta y sala, citas por estado e historial clínico.
- Panel de métricas y exportación a PDF y Excel.
- Servicio SOAP y SOAP Explorer.
- Comunicación interna entre microservicios con OpenFeign.
- Modo claro y oscuro y asistente virtual "Marca".
- Ejecución local con Docker Compose.

No incluido o no evidenciado en el proyecto:

- Despliegue en una plataforma de nube o ambiente de producción.
- Pasarela de pagos real (el pago es simulado para la demo).
- Aplicación móvil nativa.
- Integración con sistemas externos de salud o aseguradoras.

9.- PRODUCTOS Y ENTREGABLES

9.1 Aplicación Web

El producto principal es una aplicación web compuesta por un frontend Next.js y un backend de microservicios Spring Boot. Según el README, el sitio web se ejecuta en http://localhost:3000, el Gateway en http://localhost:8080 y el panel de Eureka en http://localhost:8761.

La solución se levanta con el comando docker compose up --build -d, que construye los siete módulos del backend y el frontend y crea la base de datos con datos demo. El orden de arranque es automático: MySQL y Eureka inician primero, luego los microservicios y el Gateway, y finalmente el frontend. También es posible ejecutarla sin Docker con Java 17, MySQL 8, Node 24 y pnpm.

9.2 Base de Datos MySQL

El proyecto utiliza MySQL 8.4 con cuatro bases de datos, una por dominio. Con Docker, MySQL ejecuta automáticamente los scripts de la carpeta SQL la primera vez que se crea el contenedor.

Base de datos	Usada por / contenido
fisiomarca_usuarios	AUTH-SERVICE: usuarios del sistema con su rol.
fisiomarca_catalogo	Micro-Catalogo y Micro-Soap: categorías y tabla servicios (tratamientos).
fisiomarca_reservas	Micro-Reservas: agenda, reservas, pagos, fisioterapeutas, salas e historial clínico.
fisiomarca_carrito	Micro-Carrito: carritos de los pacientes.

El script SQL/01-fisiomarca.sql crea las bases y carga un administrador, 21 pacientes, 5 categorías, 17 tratamientos, 4 fisioterapeutas, 4 salas, alrededor de 280 citas en 12 semanas, 112 notas clínicas y carritos a medio armar. El script SQL/02-estados-pagos.sql reparte pacientes en los estados Pago pendiente, Expiradas y Canceladas. El horario de la clínica es de lunes a sábado, de 07:00 a 12:00 y de 14:00 a 19:00; las fechas se guardan en UTC y el backend trabaja en hora de Lima.

9.3 Código Fuente y estructura del proyecto

El código está organizado en un repositorio con tres carpetas principales: backend, frontend y SQL, además del archivo docker-compose.yml y .env.example en la raíz. Cada módulo del backend contiene su pom.xml, su código fuente y su Dockerfile.

Componente	Contenido principal
backend/Servidor-Eureka	Servidor de registro y descubrimiento.
backend/Gateway	Rutas hacia los microservicios y validación del JWT.
backend/AUTH-SERVICE	Login, registro, usuarios y emisión del JWT.
backend/Micro-Catalogo	Categorías y tratamientos.
backend/Micro-Reservas	Agenda, reservas, pagos, equipo e historial clínico.
backend/Micro-Carrito	Carrito y conversión en reserva vía Feign.

Componente	Contenido principal
backend/Micro-Soap	Servicio SOAP contract-first del catálogo.
frontend/src/app	Rutas de Next.js (page.tsx, layouts, /api/chat).
frontend/src/vistas	Una carpeta por pantalla del sitio y del dashboard.
frontend/src/components	Componentes compartidos (navbar, admin, carrito).
frontend/src/lib	Servicios de API, sesión, tema y asistente.
frontend/src/proxy.ts	Protección de rutas por rol.

9.4 Reportes PDF y Excel

El panel de administración permite exportar la información a PDF y Excel. Para la generación de PDF se utiliza jsPDF y para Excel se utiliza ExcelJS, ambas bibliotecas integradas en el frontend Next.js.

10.- ARQUITECTURA Y TECNOLOGÍAS UTILIZADAS

La solución corresponde a una arquitectura de microservicios. El navegador accede al frontend Next.js, que envía las peticiones con el JWT al Gateway. El Gateway valida el token, consulta a Eureka la ubicación del servicio y enruta la petición al microservicio correspondiente, que a su vez se conecta a su base de datos MySQL.

10.1 Backend y microservicios

El backend está desarrollado con Java 17 y Spring Boot 4.1. Spring Data JPA e Hibernate permiten mapear las entidades de cada microservicio a su base de datos; ModelMapper facilita la conversión entre objetos y Lombok reduce el código repetitivo. Todos los servicios se registran en Eureka y el Gateway, construido con Spring Cloud Gateway WebFlux y LoadBalancer, distribuye las peticiones.

La comunicación interna se realiza con OpenFeign sin pasar por el Gateway. Micro-Reservas consulta a AUTH-SERVICE si existe el cliente y a Micro-Catalogo el precio y la duración del tratamiento. Micro-Carrito consulta a AUTH-SERVICE y a Micro-Catalogo, y llama a Micro-Reservas para convertir el carrito en reserva, reenviando el token del usuario.

Ruta en el Gateway	Microservicio
/auth/**, /usuarios/**	AUTH-SERVICE
/catalogo/**	MICRO-CATALOGO
/reservas/**	MICRO-RESERVAS
/carrito/**	MICRO-CARRITO
/ws/**	MICRO-SOAP

10.2 Frontend y presentación

El frontend está desarrollado con Next.js 16 utilizando App Router y Turbopack, React 19 y TypeScript 5. Los estilos se construyen con Tailwind CSS 4, las animaciones con Framer Motion, los gráficos con Recharts y los formularios con React Hook Form y validación con Zod. Los íconos provienen de Lucide.

Cada pantalla se ubica en frontend/src/vistas/<sección>/<nombre>/<nombre>.tsx, y los archivos page.tsx de src/app registran la URL correspondiente. Las rutas públicas son /, /servicios, /categorias, /login y /registro; las del paciente son /reservas, /reservas/nueva, /reservas/pago/{codigo}, /carrito y /perfil; y las del administrador se agrupan bajo /dashboard. El sitio ofrece modo claro y oscuro, recordado por navegador.

10.3 API REST y comunicación

Toda la comunicación entre el frontend y el backend se realiza mediante HTTP a través del Gateway en http://localhost:8080, intercambiando JSON en los servicios REST y XML en el servicio SOAP. La respuesta estándar de la API tiene la forma { "success": true, "mensaje": "...", "data": {} }.

Petición	Descripción
GET /catalogo/servicios/activos	Catálogo público de tratamientos, sin token.
POST /auth/login	Inicio de sesión; devuelve el token en data.token.
POST /auth/register	Registro de un nuevo usuario con rol CLIENTE.
GET /usuarios/perfil	Ruta protegida que requiere el token.
GET /reservas/lista	Ruta de administrador.

El detalle de los endpoints y ejemplos JSON se documenta en backend/CONTRATO_FRONTEND.md, y la explicación de cada microservicio con la colección de pruebas en Bruno se encuentra en backend/GUIA_EXPLICATIVA.md. La API también puede probarse con curl.

10.4 Seguridad

La seguridad se implementa con Spring Security, OAuth2 Resource Server y JJWT. En el registro (POST /auth/register) la contraseña se cifra con BCrypt y el usuario recibe el rol CLIENTE. En el login (POST /auth/login), AUTH-SERVICE valida las credenciales y firma un JWT con HS256 que incluye sub (email), roles, usuarioId, nombre, clienteId, iat y exp. La vigencia por defecto del token es de una hora.

El frontend envía el token en cada petición mediante la cabecera Authorization: Bearer. El Gateway valida la firma y la vigencia, devolviendo 401 si fallan; luego cada microservicio vuelve a validarlo como Resource Server y aplica los roles, devolviendo 403 si no corresponde. Esta doble validación es importante porque las restricciones del frontend por sí solas no serían suficientes para proteger la API.

Las rutas públicas son POST /auth/login, POST /auth/register, POST /usuarios/registrar, GET /catalogo/**, /ws/** y GET /usuarios/clientes/existe/**, esta última de uso interno de Feign. La clave

JWT_SECRET debe ser idéntica en AUTH-SERVICE, el Gateway, los microservicios y el frontend, y Docker Compose la comparte automáticamente.

10.5 Servicio SOAP, asistente virtual y contenerización

Micro-Soap está implementado con Spring Web Services bajo el enfoque contract-first. Su contrato se define en servicio.xsd con el namespace http://fisiomarca.com/soap, el WSDL se publica en http://localhost:8080/ws/servicio.wsdl y el endpoint es POST http://localhost:8080/ws. El servicio lee y escribe la tabla servicios de fisiomarca_catalogo mediante JdbcTemplate.

Operación	Entrada	Salida
getServiciosRequest	—	Lista de servicios.
getServicioRequest	id	Un servicio (bean).
postServicioRequest	bean	estado (filas insertadas).

El asistente virtual "Marca" funciona por defecto en modo local, con lógica propia del proyecto: botones rápidos, sugerencia de tratamiento por palabras clave y detección de emergencias. De forma opcional, con una clave de Anthropic configurada en el archivo .env, el texto libre es respondido por Claude.

Finalmente, Docker Compose levanta MySQL, los siete módulos del backend y el frontend con imágenes multi-stage. Los pagos pendientes de la demo vencen de forma escalonada y, al vencer, pasan automáticamente al estado Expiradas.

11.- CONCLUSIONES

1. Se logró desarrollar una plataforma web funcional para un centro de fisioterapia, integrando microservicios Spring Boot, MySQL y un frontend Next.js. La solución cubre el flujo completo del paciente, desde la consulta de tratamientos hasta el pago y el seguimiento de su evolución.
2. El proyecto permitió aplicar de manera práctica Service Discovery con Eureka y un API Gateway como punto de entrada único, comprendiendo cómo se localizan y enrutan los servicios en una arquitectura distribuida.
3. La implementación de JWT con Spring Security y OAuth2 Resource Server permitió aplicar una doble validación en el Gateway y en cada microservicio, diferenciando los permisos de los roles CLIENTE y ADMINISTRADOR.
4. La comunicación interna con OpenFeign permitió que los microservicios colaboren entre sí, por ejemplo para convertir el carrito en reserva, manteniendo cada dominio con su propia base de datos.
5. La integración de un servicio SOAP contract-first junto a los servicios REST permitió aplicar ambos estilos de servicios web dentro de una misma solución.
6. Las reglas de negocio sobre sesiones, citas activas, horarios apartados y expiración de pagos hacen que el sistema sea más que un CRUD básico y ayudan a mantener la consistencia de la agenda.
7. El proyecto se encuentra preparado para ejecución local con Docker Compose, de acuerdo con el README, y no se considera un despliegue en la nube dentro de los entregables actuales. Aun así, la contenerización deja una base adecuada para un despliegue posterior.

12.- RECOMENDACIONES

1. Reemplazar los valores por defecto de JWT_SECRET y de la contraseña de MySQL mediante el archivo .env o un gestor de secretos antes de cualquier despliegue.
2. Agregar pruebas automatizadas de integración para casos críticos como choques de horario, exceso de sesiones por compra, expiración de pagos y accesos sin el rol adecuado.
3. Implementar documentación formal de la API mediante OpenAPI/Swagger, complementando CONTRATO_FRONTEND.md y la colección de Bruno.
4. Integrar una pasarela de pagos real en reemplazo del pago simulado cuando el proyecto pase a un entorno productivo.
5. Realizar un despliegue controlado en una plataforma de nube aprovechando las imágenes Docker existentes, configurando correctamente CORS, variables de entorno y base de datos.
6. Incorporar notificaciones por correo o WhatsApp para recordar citas y confirmar pagos.
7. Definir un flujo de respaldo y recuperación de las cuatro bases de datos para evitar pérdida de información.

13.- GLOSARIO

Término	Definición
API REST	Interfaz que permite que aplicaciones se comuniquen mediante peticiones HTTP siguiendo recursos y operaciones definidas.
Microservicio	Componente independiente que implementa un dominio del negocio y se comunica con otros mediante la red.
Eureka	Servidor de Spring Cloud donde los microservicios se registran y se descubren entre sí.
API Gateway	Punto de entrada único que enruta las peticiones al microservicio correspondiente.
OpenFeign	Cliente HTTP declarativo de Spring Cloud usado para la comunicación entre microservicios.
JWT	JSON Web Token utilizado para representar de forma segura la información de autenticación de un usuario.
BCrypt	Algoritmo de cifrado utilizado para almacenar las contraseñas.
SOAP	Protocolo de servicios web basado en mensajes XML y en un contrato formal.
WSDL	Documento XML que describe las operaciones de un servicio SOAP.
Contract-first	Enfoque en el que primero se define el contrato (XSD/WSDL) y luego se implementa el servicio.
Spring Boot	Framework del ecosistema Spring que facilita la creación de aplicaciones Java con configuración automática.
JPA	Especificación de Java para la persistencia y el mapeo objeto-relacional.
Next.js	Framework de React para construir aplicaciones web con enrutamiento basado en archivos.

Término	Definición
Docker Compose	Herramienta que define y levanta varios contenedores con un solo comando.
Bruno	Herramienta para probar peticiones a APIs.
Escala EVA	Escala visual analógica utilizada para medir la intensidad del dolor.
CRUD	Operaciones básicas de Crear, Leer, Actualizar y Eliminar información.

14.- BIBLIOGRAFÍA

La bibliografía se presenta como apoyo técnico para las tecnologías identificadas en el proyecto.

1. Spring. (2026). Spring Boot Reference Documentation.
2. Spring. (2026). Spring Cloud Netflix, Gateway y OpenFeign Reference Documentation.
3. Spring. (2026). Spring Security Reference Documentation.
4. Spring. (2026). Spring Web Services Reference Documentation.
5. Hibernate. (2026). Hibernate ORM Documentation.
6. MySQL. (2026). MySQL Reference Manual.
7. Vercel. (2026). Next.js Documentation.
8. Tailwind CSS. (2026). Tailwind CSS Documentation.
9. Docker. (2026). Docker y Docker Compose Documentation.
10. Proyecto FisioMarca. README.md y código fuente del repositorio analizado.

15.- ANEXOS

15.1 Anexo A: Estructura principal del proyecto

Directorio	Contenido
backend/	Servidor-Eureka, Gateway, AUTH-SERVICE, Micro-Catalogo, Micro-Reservas, Micro-Carrito y Micro-Soap.
backend/CONTRATO_FRONTEND.md	Endpoints y ejemplos JSON.
backend/GUIA_EXPLICATIVA.md	Explicación de cada pieza y pruebas en Bruno.
frontend/src/	app, vistas, components, lib y proxy.ts.
frontend/public/images/	Imágenes de tratamientos y categorías.
SQL/	01-fisiomarca.sql y 02-estados-pagos.sql.
docker-compose.yml	Levanta MySQL, los 7 módulos del backend y el frontend.
.env.example	Variables opcionales de configuración.

15.2 Anexo B: Flujo general de una reserva

1. El paciente inicia sesión y AUTH-SERVICE le entrega un JWT.

2. Consulta el catálogo y añade sesiones al carrito (máximo 4 por compra).
3. Ingresa a Agendar y pagar y elige el día y la hora de cada sesión sobre la disponibilidad real (1 sesión por día).
4. Micro-Carrito valida al cliente y el tratamiento vía Feign y crea la reserva en Micro-Reservas reenviando el token.
5. Los horarios quedan apartados mientras el paciente paga con tarjeta, Yape o Plin (simulado).
6. Con el pago, la cita queda programada y aparece en Mis citas.
7. Si el pago no se completa a tiempo, la cita pasa automáticamente a Expiradas.
8. El administrador gestiona la cita desde la Agenda y registra la atención en el historial clínico.

15.3 Anexo C: Flujo de autenticación con JWT

1. El usuario se registra con POST /auth/register y su contraseña se guarda cifrada con BCrypt.
2. Inicia sesión con POST /auth/login y AUTH-SERVICE firma un JWT con HS256.
3. El frontend guarda la sesión y envía el token en la cabecera Authorization: Bearer.
4. El Gateway valida la firma y la vigencia; si falla responde 401.
5. El microservicio vuelve a validar el token y verifica el rol; si no corresponde responde 403.
6. Si todo es válido, el microservicio procesa la petición y responde con el formato estándar.

15.4 Anexo D: Evidencias en forma de captura del proyecto inicializado

- Captura de docker compose ps con los servicios en estado healthy.
  [Insertar captura]

- Captura del panel de Eureka con los servicios registrados.
  [Insertar captura]

- Captura de la página de inicio y del catálogo de tratamientos.
  [Insertar captura]

- Captura del inicio de sesión.
  [Insertar captura]

- Captura del carrito y de Agendar y pagar.
  [Insertar captura]

- Captura de Mis citas y de Perfil – Mi evolución.
  [Insertar captura]

- Captura del panel de métricas del administrador.
  [Insertar captura]

- Captura de la Agenda y de Citas por estado.
  [Insertar captura]

- Captura del historial clínico.
  [Insertar captura]

- Captura de un reporte PDF y un reporte Excel.
  [Insertar captura]

- Captura del SOAP Explorer y del WSDL.
  [Insertar captura]